

CYBR 437: Secure Coding

Secure Coding in C

Instructor: Abinash Borah

Outline

- Pointer Subterfuge
- Integer Security

Pointer Subterfuge

- Pointer subterfuge is a general term for exploits that modify a pointer's value
- Function pointers can be overwritten to transfer control to attacker-supplied shellcode – when a program executes a call via the function pointer, the shellcode is executed
- Data pointers can also be modified to run arbitrary code – if a data pointer is used as a target for a subsequent assignment, attackers can control the address to modify other memory locations
- Preventing pointer subterfuge is difficult and can be best mitigated by eliminating the initial vulnerability

Function Pointer Subterfuge

- Remember: the initialized data segment contains all initialized global and static variables
 the BSS segment contains all uninitialized global and static variables
- While stack smashing is not possible in the data segment, overwriting function pointers is equally effective in any memory segment
- A vulnerable program can be exploited to overwrite a function pointer in the BSS segment
- A naive approach to eliminate buffer overflows is to redeclare stack buffers as global or local static variables to reduce the possibility of stack-smashing attacks
- This redeclaration is inadequate because exploitable buffer overflows can occur in the data segment as well

Function Pointer Subterfuge Example

- In this program, `buff` & `funcPtr` are both uninitialized and stored in BSS
- `strncpy()` here is an unsafe use of a bounded string copy
- A buffer overflow occurs when the length of `argv[1]` exceeds `BUFSIZE`
- This buffer overflow can be exploited to transfer control to arbitrary code by overwriting the value of the function pointer with the address of the shellcode
- When the program invokes the function identified by `funcPtr`, the shellcode is invoked

```
void good_function(const char *str) {...}

int main(int argc, char *argv[]) {
    static char buff[BUFSIZE];
    static void (*funcPtr)(const char *str);
    funcPtr = &good_function;
    strncpy(buff, argv[1], strlen(argv[1]));
    (void) (*funcPtr) (argv[2]);
}
```

Data Pointer Subterfuge

- Data pointers are used to refer to dynamically allocated structures, call-by-reference function arguments, arrays, and other variables and data structures
- These data pointers can be modified by an attacker – for example, when exploiting a buffer overflow vulnerability
- If a pointer is subsequently used as a target for an assignment, an attacker can control the address to modify other memory locations – a technique known as an arbitrary memory write
- Arbitrary memory writes were of more concern on a 32-bit architecture because
 `sizeof(void *) = sizeof(int) = sizeof(long) = 4 bytes`
 So, there were more opportunities to write 4 bytes to 4 bytes to overwrite an address

Data Pointer Subterfuge Example

- `memcpy()` call is unbounded in this program
- After overflowing the buffer, an attacker can overwrite `ptr` and `val`
- When `*ptr = val` is subsequently executed, an arbitrary memory write is performed
- Data pointers can also be modified by attackers as a result of common errors in managing dynamic memory

```
void foo(void * arg, size_t len) {  
    char buff[100];  
    long val = ...;  
    long *ptr = ...;  
    memcpy(buff, arg, len);  
    *ptr = val;  
    ...  
    return;  
}
```

Modification of the Instruction Pointer

- A static invocation of a function uses an immediate value as a relative displacement for the address of the function
- This displacement cannot be overwritten because it is in the code segment
- However, the invocation of a function through the function pointer uses an indirect reference
- The address in the referenced location (typically, in the data or stack segment) can be overwritten
- These indirect function references, as well as function calls that cannot be resolved at compile time, can be exploited to transfer control to arbitrary code

```
void good_function(const char *str) {  
    printf("%s", str);  
}  
  
int main(void) {  
    static void (*funcPtr)(const char *str);  
    funcPtr = &good_function;  
    (void) (*funcPtr) ("hi");  
    good_function("there!\n");  
    return 0;  
}
```


Pointer Subterfuge Mitigation Strategies

- The best way to prevent pointer subterfuge is to eliminate the vulnerabilities that allow memory to be improperly overwritten
- Pointer subterfuge can occur as a result of overwriting data pointers, errors in managing dynamic memory, and format string vulnerabilities
- Eliminating these sources of vulnerabilities is the best way to eliminate pointer subterfuge
- Other mitigation strategies applied:
 - Use of stack canaries
 - Allowing a memory segment to be either writable or executable, but not both
 - Encoding and decoding function pointers (instead of storing a function pointer, a program can store an encrypted version of the pointer – an attacker would need to break the encryption to redirect the pointer)

Integer Security

- Integers represent an underestimated source of vulnerabilities in C programs
- Primarily because boundary conditions for integers, unlike other boundary conditions in software engineering, have often been ignored
- When developing secure systems, we cannot assume that a program will operate normally, within a given range of expected inputs – because attackers look for input values that produce an abnormal effect
- A software vulnerability may result when a program evaluates an integer to an unexpected value and then uses the value as an array index, size, or loop counter

Integer Security Example

- Here, the combined string length can exceed the capacity of an unsigned short int
- Then malloc() can allocate a buffer that is too small to hold the arguments, resulting in a buffer overflow

```
int main(int argc, char *argv[]) {  
    unsigned short int total;  
    total = strlen(argv[1]) + strlen(argv[2]) + 1;  
    char *buff = (char *)malloc(total);  
    strcpy(buff, argv[1]);  
    strcat(buff, argv[2]);  
}
```

Wraparound

- A computation involving unsigned operands can never overflow due to wraparound
- Because a result that cannot be represented by the resulting unsigned integer type is reduced
- For addition and multiplication, this is the same as pretending that there are additional high-order (most significant) bits appended to make sufficient room for the representation and then discarding these bits
- An unsigned integer expression can never evaluate to less than zero because of wraparound
- This can lead to the possibility of coding tests in C that are always true or always false
- Example: Variable `i` below cannot take a negative value, so this loop will never terminate

```
for (unsigned i = n; --i >= 0; )
```

Appropriate Testing for Wraparound

```
unsigned int i, j, sum;
if (sum + i > UINT_MAX) //cannot happen
    too_big();
else
    sum += i;
```

VS.

```
if (i > UINT_MAX - sum) // correct
    too_big();
else
    sum += i;
```

```
unsigned int i, j, sum;
if (sum - j < 0) // cannot happen
    negative();
else
    sum -= j;
```

VS.

```
if (j > sum) // correct
    negative();
else
    sum -= j;
```

Integer Overflow

- An integer overflow occurs when the value of an integer is increased beyond its maximum value or decreased beyond its minimum value
- Overflows can occur with signed or unsigned integers
 - A signed overflow occurs when a value is carried over to the sign bit
 - An unsigned overflow occurs when the underlying representation can no longer represent a value

Integer Conversion and Truncation Errors

- Conversion is a change in the underlying data type used to represent a value resulting from assignment, type casting, or computation
- Conversion from a type with a smaller size (number of bytes) to a type with a larger size generally preserves the mathematical value
- However, conversion in the opposite direction can easily cause loss of high-order bits (worse when signed integer types are involved) – unless the magnitude of the value is small enough to be represented correctly
- Conversions occur explicitly as the result of a cast or implicitly as required by an operation – for example, implicit conversions occur when operations are performed on mixed types or when a value needs to be converted to an appropriate argument type
- While implicit conversions simplify programming, they can also lead to lost or misinterpreted data

Integer Conversion and Truncation Errors (contd.)

- The C Standard rules define how C compilers handle conversions
- These rules include
 - Integer conversion rank,
 - Integer promotions, and
 - Usual arithmetic conversions

Integer conversion rank

- Every integer type has an integer conversion rank that determines how conversions are performed
- Integer conversion rank provides a standard rank ordering of integer types that is used to determine a common type for computations

Integer Conversion and Truncation Errors (contd.)

Integer promotions

- A data object or expression with an integer type whose integer conversion rank is less than or equal to the rank of int and unsigned int is promoted when used in an expression where an int or unsigned int is required
- The integer promotions are applied as part of the usual arithmetic conversions
- Example:
 - Integer promotions rule requires the promotion of the values of variables c1 and c2 to int
 - The two values of type int are added, yielding a value of type int, and the result is stored in the int variable sum

```
int sum;  
char c1, c2;  
sum = c1 + c2;
```

Integer Conversion and Truncation Errors (contd.)

Integer promotions (contd.)

- Integer promotions are also applied to avoid arithmetic errors resulting from the overflow of intermediate values and to perform operations in a natural size for the architecture
- In the following code segment, the value of c1 is multiplied by the value of c2, and the product is divided by the value of c3 according to operator precedence rules
- If the expression is evaluated in this order, without integer promotions, the multiplication of c1 and c2 results in an overflow
- With integer promotions, c1, c2, and c3 are converted to int; result is truncated and stored

```
signed char result, c1, c2, c3;  
c1 = 100;  
c2 = 3;  
c3 = 4;  
result = c1 * c2 / c3;
```

Integer Conversion and Truncation Errors (contd.)

Usual Arithmetic Conversions

- The usual arithmetic conversions are a set of rules that provide a mechanism to yield a common type for the operands
- Balancing conversions involves two operands of different types – one or both operands may be converted
- Conversion of a value in an unsigned integer type to a smaller size is accomplished by truncating the larger value and preserving the low-order bits – data is lost if the value cannot be represented in the new type
- If the programmer does not anticipate this possibility, a programming error or vulnerability can occur
- Conversions that occur between signed and unsigned integer types of any size can result in lost or misinterpreted data when a value cannot be represented in the new type

Integer Conversion and Truncation Errors (contd.)

Conversion Implications

- Implicit conversions simplify programming in C
- However, conversions have the potential for lost or misinterpreted data
- Programmers should avoid conversions resulting in:
 - Loss of value: conversion to a type where the magnitude of the value cannot be represented
 - Loss of sign: conversion from a signed type to an unsigned type that results in loss of sign
- The only integer type conversion guaranteed to be safe for all data values and all conforming implementations is to a larger type of the same signedness

Integer Operations

- Integer operations can result in exception condition errors such as overflow, wrapping, and truncation
- Exception conditions occur when the result of an operation cannot be represented in the type resulting from the operation
- It is particularly important to secure operations on integer values that originate from untrusted sources and are used in any of the following ways:
 - As an array index
 - In pointer arithmetic
 - As the length or size of an object
 - As the bounds of an array (for example, a loop counter)
 - As an argument to a memory allocation function
 - In security-critical code

Mitigation Strategies for Integer Vulnerabilities

Range Checking

- The burden for integer range checking in C is mostly left to the programmer
- Providing range checks for all operations can be problematic because a typical program can have many such operations – checking all of them could be slow and use more memory
- Range errors resulting in integer values that are used in allocating or accessing memory are more likely to result in a security vulnerability than are integers used for other purposes
- One way to limit the number of tests that need to be performed is to restrict the input of integer values to ranges that could never result in an out-of-range integer value
- All external inputs should be evaluated to determine whether there are identifiable upper and lower bounds, and if so, these limits should be enforced by the interface

Mitigation Strategies for Integer Vulnerabilities (contd.)

Better Type Selection

- One way to provide better type checking is to use better types
- Using an unsigned type can guarantee that a variable does not contain a negative value – this solution does not prevent overflow
- Better type selection should be used so that the compiler can be more effective in identifying range problems – e.g., short int may not be adequate for an array size

Compiler Runtime Checks

- GCC compiler includes an -ftrapv compiler option that provides limited support for detecting integer exceptions at runtime
- This option generates traps for signed overflow on addition, subtraction, and multiplication operations

Mitigation Strategies for Integer Vulnerabilities (contd.)

Safe Integer Operations

- An additional approach for preventing integer errors is to protect each operation
- This approach can be labor-intensive and expensive to perform
- A solution is using secure integer libraries for all operations on integers, where one or more of the inputs could be influenced by an untrusted source
- Secure libraries ensure that integer operations either succeed or report an error
- To use these secure libraries, code must be specifically developed to invoke secure integer functions rather than relying on built-in operators